Schedule Calculation — Algorithm Analysis

Topological Sort on the Module Prerequisite Graph

---

1. Problem Definition

A degree programme consists of study modules, some of which carry *prerequisites* — other modules that must be completed in an earlier semester. Assigning every module to a semester that respects all prerequisite constraints is an instance of the classical **topological sort** problem on a **directed acyclic graph** (DAG).

Let $G = (V, E)$ be a directed graph where

- V is the finite set of study modules (vertices), and
- $(u, v) \in E$ means module u must be completed before module v (a prerequisite edge).

A **valid study schedule** is a partition of V into ordered layers $L_1, L_2, ..., L_k$ such that for every edge $(u, v) \in E$:

$$\text{layer}(u) < \text{layer}(v)$$

The earliest semester in which a module v can be placed equals one more than the length of its longest incoming prerequisite chain:

$$\text{semester}(v) = 1 + \max_{u : u \to v} \text{semester}(u)$$

where the maximum is taken over all direct prerequisites of v in G, and $\text{semester}(v) = 1$ when v has no prerequisites. This quantity is also known as the **critical-path depth** of the vertex in the DAG.

2. Kahn's Layered BFS

Kahn's algorithm computes a layered topological ordering in $O(|V| + |E|)$ time. It maintains three auxiliary data structures built from G:

`moduleMap` Maps each module key to its `StudyModule` object for $O(1)$ retrieval by key.

`inDegree` Maps each module key to its current **in-degree** — the number of its prerequisites that have not yet been placed. A module is ready to be scheduled exactly when its in-degree reaches zero.

`dependents` The **reverse adjacency list** of G: maps each module key to the list of modules that declare it as a prerequisite. When a module is placed, its entry in `dependents` tells the algorithm which in-degrees to decrement, at $O(1)$ per edge.

```
BUILD moduleMap    ← { key → StudyModule }                    | O(V)
BUILD inDegree     ← { key → count of in-program prereqs }    | O(V + E)
BUILD dependents   ← reverse adjacency list                   | O(V + E)
                                                              |
ready ← { v ∈ V | inDegree[v] = 0 }                           | O(V)
placed ← 0,  semesterNumber ← 1

WHILE ready ≠ ∅                                              ┐
  layer ← snapshot(ready)                                    |
  ready ← ∅                                                  | each vertex
  semester ← new Semester(semesterNumber++)                  | dequeued
  FOR EACH key ∈ layer                                       | once  → O(V)
    semester.add(moduleMap[key]);  placed++                  |
    FOR EACH dep ∈ dependents[key]                           | each edge
      inDegree[dep] ← inDegree[dep] − 1                      | traversed
      IF inDegree[dep] = 0 THEN ready.add(dep)               | once  → O(E)
  schedule.addSemester(semester)                             |
END WHILE                                                    ┘

IF placed < |V| THEN throw CyclicDependencyException
RETURN schedule
```

Listing 1: Kahn's layered BFS — pseudocode with per-step cost annotations

The critical correctness invariant is the **layer snapshot**: before processing a layer, the ready queue is copied into `layer` and immediately cleared. Any module whose in-degree reaches zero during iteration i is added to the now-empty queue and will only be dequeued in iteration $i+1$, ensuring it is placed in the **next** semester rather than the current one.

3. Complexity Analysis

Step	Cost	Reason
`buildModuleMap`	$O(V)$	One map insertion per module.
`buildInDegree`	$O(V+E)$	Iterate all V modules; for each, scan its prerequisite list to count in-program edges.
`buildDependents`	$O(V+E)$	Iterate all E edges once to populate the reverse adjacency list.
Seed ready queue	$O(V)$	Single pass over all V entries of `inDegree`.
BFS main loop	$O(V+E)$	Each vertex enters and leaves the queue exactly once: $O(V)$. Each edge (u, v) is visited exactly once when u is dequeued: $O(E)$.
Cycle detection	$O(V)$	Single pass over `inDegree` to collect entries with a positive value.

Step	Cost	Reason
Total	$\boldsymbol{O(V+E)}$	All individual steps are bounded by the BFS traversal.

4. Cycle Detection

A valid degree programme must form a DAG. A cycle would imply a group of modules each depending on another within the group, making it impossible to place any of them in any finite schedule.

Kahn's algorithm detects cycles **without additional work**: if the BFS terminates before all modules have been placed — that is, `placed` $< |V|$ — the modules still carrying a positive in-degree are exactly those involved in or directly blocked by a cycle. Their keys are collected and reported in a `CyclicDependencyException`.

This follows directly from the correctness theorem for Kahn's algorithm:

G has a valid topological order if and only if Kahn's BFS places every vertex.

If G is a DAG, every vertex eventually reaches in-degree zero and is placed. If G contains a cycle $C \subseteq V$, the vertices of C mutually block each other: no vertex in C can ever reach in-degree zero (doing so would require one of its predecessors in C to have been placed first, but that predecessor is itself blocked), so the BFS drains while the cycle vertices remain unplaced.